caches, or optimizing data structures. At the other extreme, it may make sense to use a detached thread where there's another mechanism for identifying when the thread has completed or where the thread is used for a "fire and forget" task.

As you've already seen in section 2.1.2, you detach a thread by calling the `detach()` member function of the `std::thread` object. After the call completes, the `std::thread` object is no longer associated with the actual thread of execution and is therefore no longer joinable:

```
std::thread t(do_background_work);
t.detach();
assert(!t.joinable());
```

In order to detach the thread from a `std::thread` object, there must be a thread to detach: you can't call `detach()` on a `std::thread` object with no associated thread of execution. This is exactly the same requirement as for `join()`, and you can check it in exactly the same way—you can only call `t.detach()` for a `std::thread` object `t` when `t.joinable()` returns `true`.

Consider an application such as a word processor that can edit multiple documents at once. There are many ways to handle this, both at the UI level and internally. One way that seems to be increasingly common at the moment is to have multiple independent top-level windows, one for each document being edited. Although these windows appear to be completely independent, each with its own menus and so forth, they're running within the same instance of the application. One way to handle this internally is to run each document-editing window in its own thread; each thread runs the same code but with different data relating to the document being edited and the corresponding window properties. Opening a new document therefore requires starting a new thread. The thread handling the request isn't going to care about waiting for that other thread to finish, because it's working on an unrelated document, so this makes it a prime candidate for running a detached thread.

The following listing shows a simple code outline for this approach.